LESSON 05 · MCP · 50 MIN

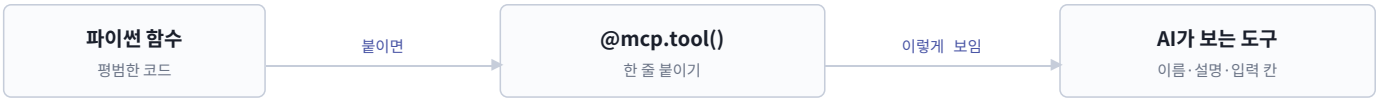
5교시 내 API를 AI에게 열어주기

새로 배울 게 많지 않습니다. 2교시에 만든 API를 부르는 함수를 쓰고, 그 위에 **한 줄만 붙이면** 끝입니다. 다만 AI는 코드를 읽지 않고 **설명문**을 읽는다는 점이 중요합니다.

설명 10분 · 따라하기 30분 · 정리와 질문 10분

OVERVIEW

내 함수가 AI의 도구가 되기까지



> 2교시의 @app.get 과 생김새가 비슷하죠. 그때는 **주소**가 생겼고 이번에는 **도구**가 생깁니다. 함수 위에 한 줄 붙이는 방식은 똑같습니다.

WORDS

새로 나오는 말

말	무슨 뜻인가	언제 나오나
`@mcp.tool()`	이 함수를 도구로 만든다	함수 바로 위에 붙인다
함수 이름	도구의 이름이 된다	AI가 이 이름으로 부른다
타입 힌트 `(order_id: int)`	입력 칸이 된다	숫자 칸인지 글자 칸인지 정해진다
설명문 `(""" """)`	AI가 읽는 사용 안내	여기가 가장 중요하다
함수 안의 코드	실제 하는 일	AI에게는 보이지 않는다
`mcp dev`	Inspector 로 확인	만들면서 바로 테스트
권한	붙인 도구만큼 할 수 있다	위험한 건 안 붙이는 게 안전
`httpx`	다른 서버를 부르는 도구	2교시 API를 부를 때 사용

FILES

실습용 파일 — 그대로 옮겨 적으세요

server.py

```
import httpx
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("내주문")

BASE = "http://127.0.0.1:8000"


@mcp.tool()
def list_orders() -> list:
    """주문 목록을 전부 보여줍니다.

    아무것도 바꾸지 않고 보기만 합니다.
    """
    r = httpx.get(f"{BASE}/orders", timeout=5.0)
    return r.json()


@mcp.tool()
def get_order(order_id: int) -> dict:
    """주문 번호 하나로 그 주문을 조회합니다.

    번호를 이미 알고 있을 때 사용하세요.
    목록 전체가 필요하면 list_orders 를 쓰세요.
    """
    r = httpx.get(f"{BASE}/orders/{order_id}", timeout=5.0)
    if r.status_code == 404:
        raise ValueError(f"{order_id}번 주문이 없습니다")
    return r.json()


if __name__ == "__main__":
    mcp.run()
```

STEPS

실습 순서

STEP 01 준비 — 2교시 서버 켜기

도구가 부를 대상을 살려 둡니다.

```
# my-api 폴더에서 (가상환경 활성화 상태로)
cd my-api
uvicorn main:app --reload

# 브라우저로 확인
http://127.0.0.1:8000/orders
```

확인 포인트

- [ ] 주문 목록이 나온다

> **이 서버를 끄면 오늘 만들 도구가 아무 일도 못 합니다.** 켜 둔 채로 새 터미널을 여세요.

STEP 02 필요한 것 설치

MCP 만들기 도구를 깝니다.

```
# 새 터미널에서 같은 폴더, 가상환경 활성화 후
pip install "mcp[cli]" httpx
```

확인 포인트

- [ ] 설치가 오류 없이 끝난다

> 설치가 오래 걸릴 수 있습니다. 끝날 때까지 기다리세요.

STEP 03 도구 두 개 만들기

함수 위에 한 줄 붙여 도구로 만듭니다.

```
# 위 '실습용 파일'의 server.py 를 만듭니다
# 천천히 보면서 세 부분을 확인하세요

#  ① @mcp.tool()        ← 도구로 만드는 줄
#  ② def list_orders()  ← 도구 이름이 됨
#  ③ """ ... """       ← AI가 읽는 설명
```

확인 포인트

- [ ] 파일이 만들어졌다
- [ ] 세 부분이 어디인지 손으로 짚을 수 있다

> 함수 안의 httpx 부분은 **2교시 API를 부르는 코드**입니다. 이해 안 돼도 괜찮아요 — 오늘 중요한 건 위의 세 부분입니다.

STEP 04 Inspector 로 확인하기

AI 눈에 어떻게 보이는지 봅니다.

```
npx -y @modelcontextprotocol/inspector python server.py

# 브라우저가 열리면 Connect → Tools 탭
```

확인 포인트

- [ ] 도구 두 개가 보인다 (list_orders, get_order)
- [ ] **내가 쓴 설명문**이 그대로 설명에 들어가 있다
- [ ] get_order 에는 order_id 라는 **숫자 입력 칸**이 생겼다

> 스키마 같은 걸 따로 쓴 적이 없죠? **타입 힌트와 설명문이 그대로 인터페이스가 됩니다.**
> 그래서 이 둘을 대충 쓰면 그대로 품질이 떨어져요.

STEP 05 직접 실행해 보기

도구가 진짜 동작하는지 확인합니다.

```
# Tools 탭에서 list_orders 선택 → Run Tool
# get_order 선택 → order_id 에 7 입력 → Run Tool
# order_id 에 999 입력 → Run Tool
```

확인 포인트

- [ ] list_orders 는 목록이 나온다
- [ ] 7은 주문 하나가 나온다
- [ ] 999는 "999번 주문이 없습니다" 라는 메시지가 나온다

> 오류 메시지도 **AI에게 그대로 전달됩니다.** "실패" 라고만 쓰면 AI는 다음에 뭘 해야 할지 모릅니다. 무엇이 잘못됐는지 적어 주세요.

STEP 06 설명을 망가뜨려 보기 — 오늘의 핵심

설명이 왜 중요한지 직접 겪습니다.

```
# 두 함수의 설명문을 아래처럼 바꿔 봅니다
"""주문을 다룹니다."""
"""주문을 다룹니다."""

# Inspector 를 껐다 다시 실행
# Tools 탭에서 두 도구를 비교해 보세요
```

확인 포인트

- [ ] 두 도구의 설명이 **똑같아졌다**
- [ ] 설명만 봐서는 어느 걸 써야 할지 알 수 없다

> AI도 똑같이 헷갈립니다. 만약 도구 중 하나가 **주문 취소** 였다면? "주문 보여줘" 라고 했는데 취소될 수도 있습니다.
> **코드는 완벽했는데 설명만 대충 써서 사고가 나는 것**입니다.

STEP 07 설명 되돌리고 좋은 설명 쓰기

어떻게 써야 하는지 익힙니다.

```
# 원래 설명으로 되돌리고, 아래 세 가지가 들어있는지 확인
#  ① 무엇을 하는지
#  ② 언제 쓰는지
#  ③ 언제 쓰지 않는지 (다른 도구를 가리키기)

"""주문 번호 하나로 그 주문을 조회합니다.

번호를 이미 알고 있을 때 사용하세요.
목록 전체가 필요하면 list_orders 를 쓰세요.
"""
```

확인 포인트

- [ ] 설명에 세 가지가 모두 들어 있다
- [ ] Inspector 에서 두 도구가 확실히 구분된다

> 특히 **③번이 중요합니다.** "이럴 땐 저 도구를 쓰세요" 라는 문장이 도구 사이에 경계를 만들어 줍니다.

STEP 08 AI에게 붙이고 물어보기

실제로 대화 속에서 써 봅니다.

```
# 4교시에서 한 것처럼 클라이언트 설정에 추가
#  command: python (또는 가상환경의 python 전체 경로)
#  args: ["server.py 의 전체 경로"]

# 프로그램을 껐다 켠 뒤
"주문 목록 보여줘"
"7번 주문 알려줘"
```

확인 포인트

- [ ] AI가 도구를 불러 실제 주문 데이터를 답한다
- [ ] 2교시에 내가 만든 데이터가 AI 대화에 나온다

> **여기가 오늘의 도착점입니다.** 2교시에 만든 내 API를 5교시에 AI가 쓰고 있어요.
> 설정이 안 되면 Inspector 로 확인한 것만으로도 충분합니다.

STEP 09 위험한 도구 이야기

무엇을 붙이면 안 되는지 감을 잡습니다.

```
# 만약 이런 도구를 붙였다면?
@mcp.tool()
def delete_all_orders():
    """모든 주문을 삭제합니다."""
    ...

# 붙이지 마세요. 오늘은 만들지 않습니다.
```

확인 포인트

- [ ] 왜 위험한지 설명할 수 있다

> AI는 자기가 읽은 문서 안에 "모두 삭제해" 같은 문장이 숨어 있어도 **그게 지시인지 그냥 글자인지 구분하지 못합니다.**
> 그래서 가장 확실한 방법은 **위험한 도구를 아예 안 만드는 것**입니다. 보기만 하는 도구부터 시작하세요.

WHEN STUCK

막혔을 때 보는 곳

증상	원인	해결
도구 호출이 실패함	2교시 서버가 꺼져 있음	uvicorn main:app --reload 가 돌고 있는지 확인
Inspector 에 도구가 안 보임	@mcp.tool() 을 안 붙임	함수 바로 위에 붙었는지 확인 · 괄호 () 도 필요
설명이 비어 있음	설명문을 안 씀	함수 첫 줄에 따옴표 세 개로 설명 작성
클라이언트에서 python 을 못 찾음	가상환경 경로 문제	가상환경 안 python 의 전체 경로를 설정에 적기
한글 설명이 깨짐	파일 인코딩	편집기에서 UTF-8 로 저장

CHECK

스스로 확인하기 — 답할 수 있으면 통과

- [ ] 함수를 도구로 만드는 한 줄은 무엇인가요?
- [ ] AI는 코드와 설명 중 무엇을 읽나요?
- [ ] 타입 힌트(order_id: int)는 AI에게 무엇이 되나요?
- [ ] 좋은 설명에 들어가야 할 세 가지는?
- [ ] 위험한 도구를 다루는 가장 확실한 방법은?

한 줄 정리

AI는 이름·설명·입력 칸 세 가지만 봅니다. 그래서 설명문이 코드만큼 중요합니다.

다음 시간 → 6교시 — 할 수는 있는데 잘 못할 때

WORKSHEET

막힌 STEP 번호 · 화면에 나온 빨간 글씨 · 강사에게 물어볼 것